

ΠΡΟΣΟΜΟΙΩΣΗ ΑΡΧΙΤΕΚΤΟΝΙΚΗΣ (Simulation)

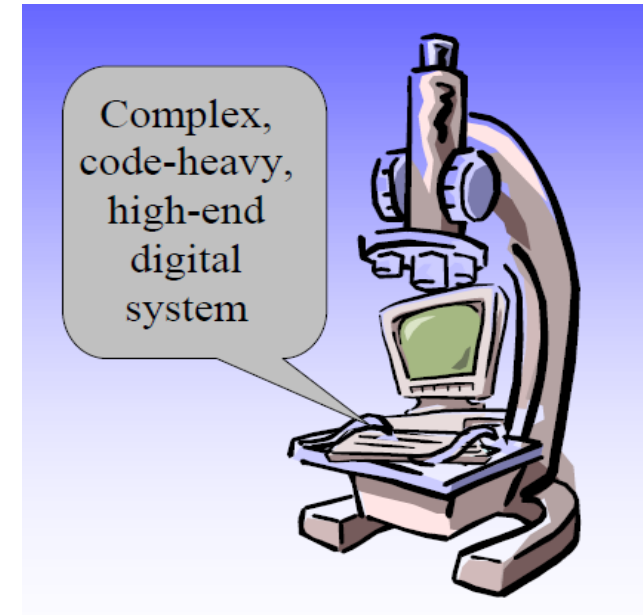


Πειραματική Αξιολόγηση Ιδέας

- *Πως θα βελτιώσει τη μετρική X το καινούριο component Y ?*
- Η απάντηση είναι workload-dependent
 - e.g., pipeling, caching, etc.
- Οι εφαρμογές αλλάζουν...
- Πληθώρα σχεδιαστικών επιλογών και παραμέτρων
 - Ο αρχιτέκτονας πρέπει να κάνει αμέτρητες επιλογές κατά το σχεδιασμό του συστήματος
 - Πολύ δύσκολη (αν όχι απίθανη) η αξιολόγηση όλων των δυνατών συνδυασμών
- Οι παράμετροι του συστήματος αλλάζουν

Πολυπλοκότητα Συστήματος

- Processor Design
 - Single thread → pipeline, branch prediction
 - Multiple threads → SMT resource allocation, threads scheduling
- Ετερογενείς αρχιτεκτονικές
 - ARM (big.LITTLE), Intel E-Cores and P-Cores
 - NVIDIA Superchip (Grace + Hopper/Blackwell)
- Ιεραρχία μνήμης
 - Multiple levels, NUMA, cache sharing
 - Coherence Protocols
- Παράλληλα συστήματα
 - Coherence, Δίκτυα διασύνδεσης
 - Compilers, automatic parallelization
 - Programming Models, synchronization costs, locks, computation and communication overheads



Υπάρχοντα Συστήματα

- Μεγάλο κόστος
 - Oracle SPARC Enterprise T5120 server ($2 \times 16 \times 8 = 256$ threads, 512GB mem) ~ 64K \$ (2014)
 - Oracle SPARC Enterprise T8-1 server ($32 \times 8 = 256$ threads, 1TB mem, 8 Data Analytics Accelerators) ~ ??K \$ (2017)
 - Intel Xeon Gold 6138 (2×20 cores, 80 threads, 768GB mem) ~ 19K \$ (2018)
 - AMD EPYC 9654 “Genoa server (2x 96 cores, 2.25GB L3, 6TB DDR5) ~30K \$ (2022)
 - Apple M2 Ultra / M3 Max Systems (24 cores, 76-core GPU, 192GB unified memory) ~7K \$ (2023)
 - NVIDIA Grace Hopper server
 - » 1 GH100 Superchip ~ 50K \$ (2024)
 - » Intel Sapphire Rapids CPUs + 8x H100 GPUs, 640GB HBM3 ~ 400K \$ (2023)
 - NVIDIA GB200 NVL72 (36 CPURs, 72 GPUs) ~ 3M (2025)

Υπάρχοντα Συστήματα

- Μεγάλο κόστος → Οικονομικά απαγορευτική πρόσβαση σε *cutting-edge* hardware
- Αδυναμία παρέμβασης στο υλικό τους
 - pipeline, caches, interconnection network
- Περιορισμένη δυνατότητα παρακολούθησης και μετρήσεων
 - performance counters → λίγοι, μικρό documentation
- Περιορισμός στο σήμερα
 - μελλοντικές αρχιτεκτονικές (π.χ. chip με 100 ή 1000 threads) ;

Τρόποι πειραματικής αξιολόγησης

- Αποδείξεις (theoretical proof)
- Αναλυτικά μοντέλα (analytical modeling/estimation)
- Προσομοίωση (simulation)
- Prototyping (e.g., FPGAs)
- Πραγματική υλοποίηση

Τι είναι ο simulator;

- Ένα εργαλείο που αναπαράγει τη «συμπεριφορά» ενός υπολογιστικού συστήματος.
- Σημαντικό εργαλείο για τον αρχιτέκτονα
 - Πληροφορίες σχετικά με την εσωτερική λειτουργία (Performance Analysis)
 - Δυνατότητα ανάπτυξης λογισμικού για μη διαθέσιμες (ή και μη υπαρκτές) πλατφόρμες
 - Προβλέψεις απόδοσης για διαφορετικές αρχιτεκτονικές
- **Dream → Create → Simulate → Evaluate**
 - Εξερεύνηση ονείρων/ιδεών
 - Έλεγχος πραγματικότητας
 - Απόφαση ποιο όνειρο/ποια ιδέα είναι καλύτερο/καλύτερη
 - **ΑΛΛΑ: Μπορεί να σε κοροϊδέψει να αποδεχτείς μια λάθος ιδέα**

Στόχοι Προσομοίωσης

- **Design space exploration**

- Επεξεργαστές/Πλατφόρμες επόμενης γενιάς
- Νέες ιδέες, components, etc.
- **Αξιολόγηση των συνεπειών/αποτελεσμάτων καινούριων σχεδιαστικών επιλογών**

- **Προσομοίωση συμπεριφοράς υπάρχοντος συστήματος**

- Cycle-level accurate debug & verification
- Αξιολόγηση μικρών βελτιώσεων (tweaks) του συστήματος (π.χ. ενέργεια, επίδοση)
- **Υψηλή ακρίβεια**

Στάδια σχεδιασμού & αξιολόγησης συστήματος

- High-level models (abstract, C)
-
- Medium-level models
- ...
- Low-level models (RTL)
- ...
- Real design

Στάδια σχεδιασμού & αξιολόγησης συστήματος

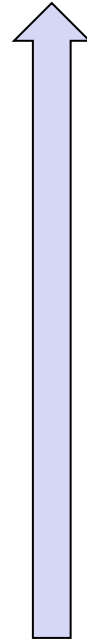
- High-level models (abstract, C)
-
- Medium-level models
- ...
- Low-level models (RTL)
- ...
- Real design



- Abstraction ↓
- Accuracy ↑
- Flexibility ↓
- Speed ↓

Στάδια σχεδιασμού & αξιολόγησης συστήματος

- High-level models (abstract, C)
-
- Medium-level models
- ...
- Low-level models (RTL)
- ...
- Real design



- Με βάση το πραγματικό σύστημα μπορώ να γυρίσω πίσω και να βελτιώσω την ακρίβεια των μοντέλων κτλ.

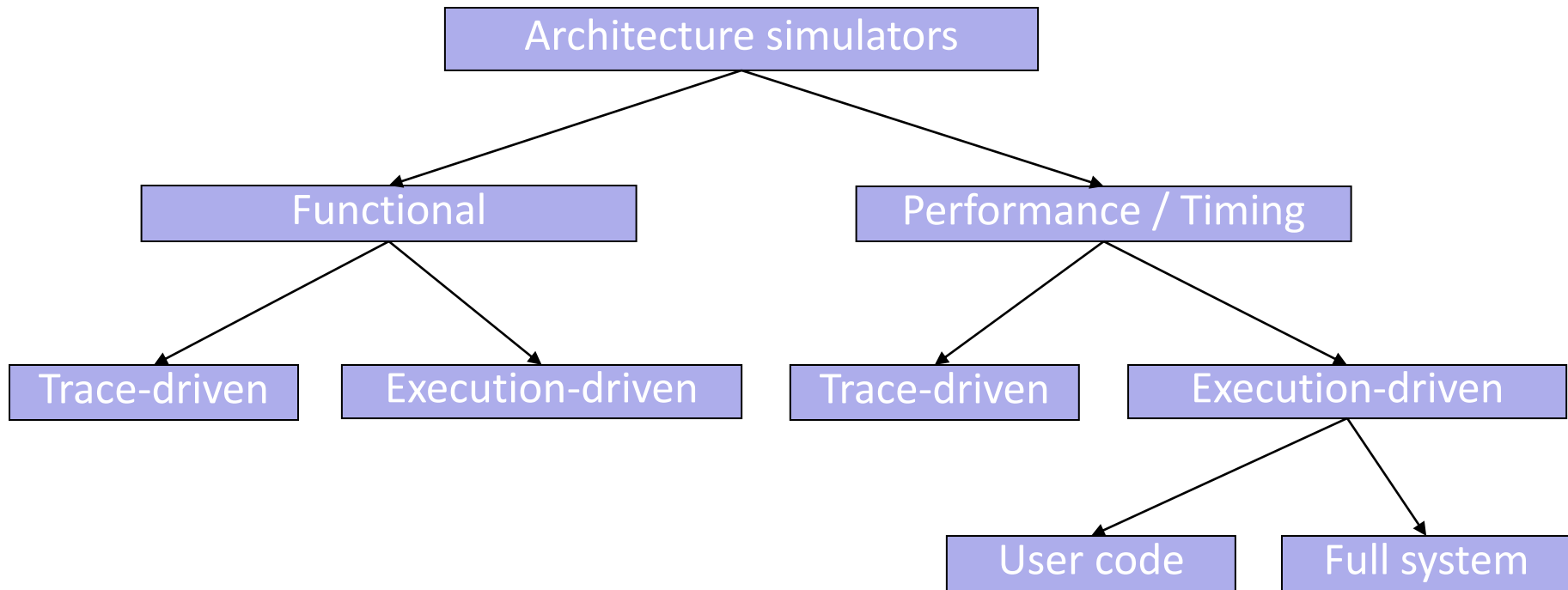
RTL Simulation

- Όχι κατάλληλη για design space exploration
 - Πολύ αργή (υλοποίηση)
 - Πολύ αργή (εκτέλεση)
 - Πολλά workloads
 - Πολλές σχεδιαστικές επιλογές/παράμετροι
 - » Cache size, associativity, block size, replacement policies
 - » Memory control, scheduling
 - » In-order / out-of-order execution, reservation stations, ROB size, etc.
 - » ...
- **High-Level Simulation**

Ιδιότητες προσομοίωσης

- Τρεις κύριες μετρικές
 - **Ταχύτητα** (speed): Πόσα γρήγορα τρέχει η προσομοίωση (MIPS)
 - **Ευελιξία** (flexibility): Πόσο γρήγορα μπορεί να υλοποιηθεί μια καινούρια πολιτική ή ένα καινούριο component?
 - **Ακρίβεια** (accuracy): Πόσο ακριβή είναι τα αποτελέσματα;
- **High-Level simulation**
 - Θυσιάζει ακρίβεια – Κερδίζει ταχύτητα & ευελιξία
 - Γρήγορη μοντελοποίηση (e.g., omit corner cases)
 - Μας ενδιαφέρουν οι τάσεις (trends) και όχι τα ακριβή νούμερα
 - **ΑΛΛΑ**: Πιθανώς λάθος αποφάσεις
 - » Πως μπορούμε να βρούμε τις σχετικές τάσεις (relative trends) με ικανοποιητική ακρίβεια;

Simulator taxonomy



Functional vs. Timing Simulators

- Functional Simulators

- Visible architectural state
- Προσομοίωση της λειτουργικότητας των εντολών (instructions semantics and functionality), μεταβολή του state (registers, memory)
- Σωστό program output
- Κύριος σκοπός: Software development and/or emulation

- Timing Simulators

- Microarchitecture details
- Λεπτομερής υλοποίηση των διαφορετικών δομών (pipeline, branch predictors, interconnection networks, memory hierarchy, etc.)
- Χρονισμός γεγονότων, προκειμένου να υπολογισθεί ο χρόνος εκτέλεσης του προγράμματος

- Functional simulation πολύ πιο γρήγορο

Trace vs. Execution-driven Simulators

- Trace Simulators
 - Εκτέλεση της εφαρμογής σε πραγματική πλατφόρμα → trace (instruction, address, ...)
 - Τα traces χρησιμοποιούνται σαν inputs του simulator
- Execution-driven Simulators
 - Ο simulator εκτελεί την εφαρμογή
 - Διατήρηση application state και architecture state
- Trace-driven simulation συνήθως πιο γρήγορο
 - Διατήρηση μόνο του architecture state, δεν εκτελούνται όλοι οι υπολογισμοί
- Τα traces επιτρέπουν την προσομοίωση proprietary applications & input sets.
- **Πρόβλημα**: Τα traces δεν μπορούν να συλλάβουν/φανερώσουν την πλήρη δυναμική συμπεριφορά της εφαρμογής

User code vs. Full system simulators

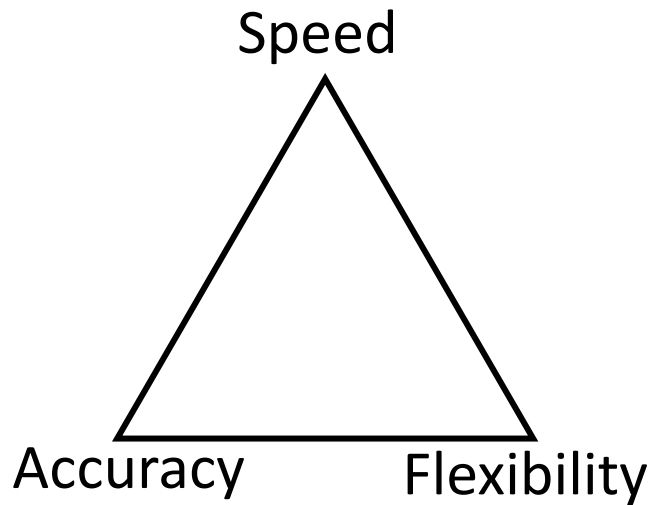
- User code Simulators

- Προσομοίωση μόνο του κώδικα της εφαρμογής
- System calls και I/O εκτελούνται με functional simulation
- Συνήθως το functional emulation πραγματοποιείται από το host OS
 - » host OS = target OS

- Full system Simulators

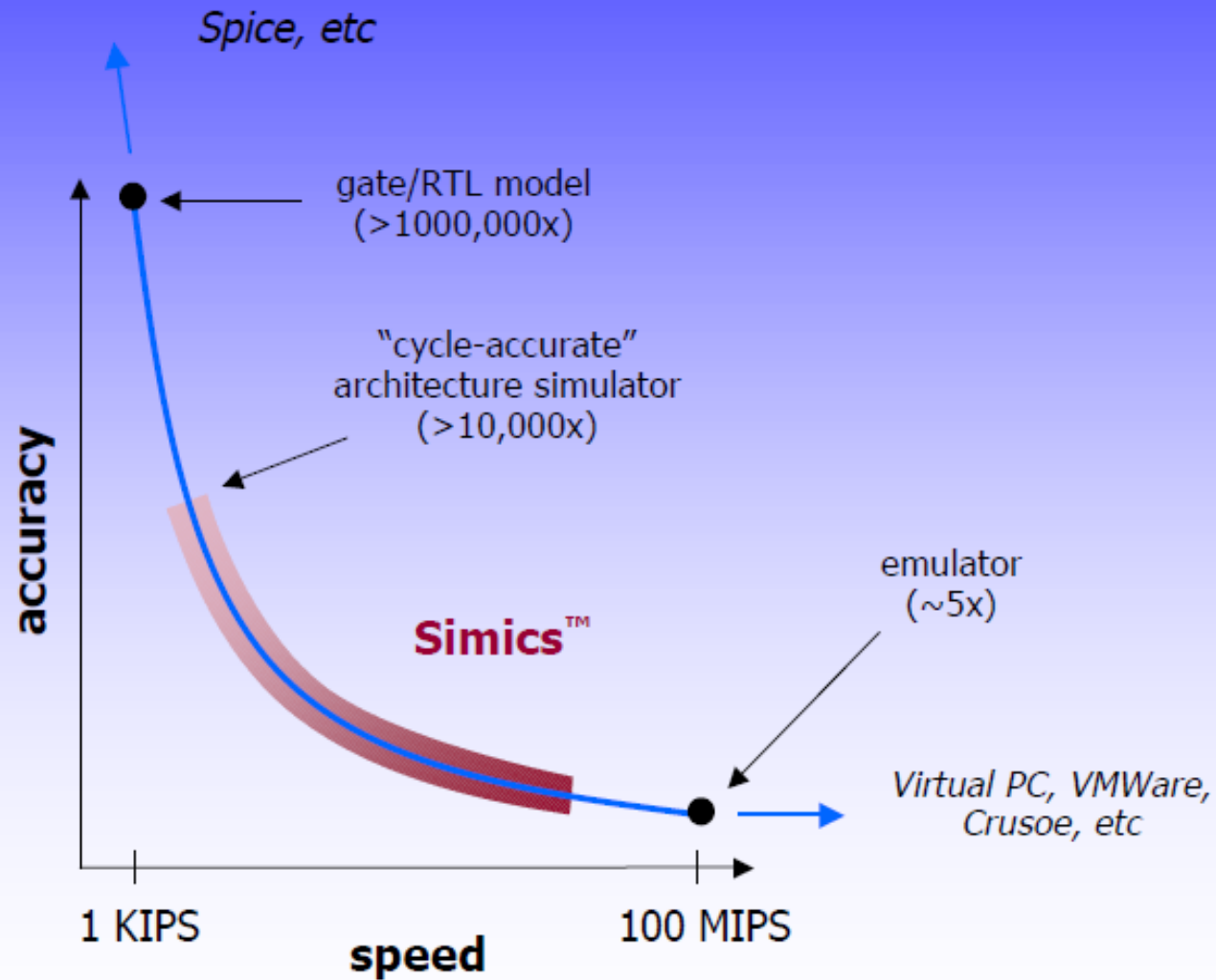
- Προσομοίωση της εφαρμογής
- Προσομοίωση του OS
- Προσομοίωση των devices (disks, network, etc.)

The Zen of architecture simulators



- Δεν υπάρχει ο τέλειος simulator
- FPGA prototypes
 - Γρήγορα, ακριβή, αλλά έλλειψη ευελιξίας
- Λεπτομερή software μοντέλα
 - Ακριβή, ευέλικτα, αλλά αργά
- Αφηρημένα software μοντέλα
 - Γρήγορα, ευέλικτα αλλά όχι ακριβή

Speed vs. Accuracy



Παράδειγμα χρόνων προσομοίωσης (1)

- spec2k with gcc and small inputs

	Time	Ratio to Native	Ratio to Functional
Native	1,054s	--	--
sim-fast	2m 47s	158x	1
sim-outorder	1h 11m 07s	4,029x	25x
simics	7m 41s	437x	1
w/Ruby	11h 27m 25s	39,131x	89x
w/Ruby + Opal	43h 13m 41s	147,648x	338x

Παράδειγμα χρόνων προσομοίωσης (2)

- Οι πραγματικές εφαρμογές παίρνουν ώρες σε πραγματικά μηχανήματα
- Χρειαζόμαστε αρκετά μεγάλες ταχύτητες για να τρέξουμε ένα σημαντικό κομμάτι αυτών των εφαρμογών.

	Number of Ops (B)
Windows XP Boot	5
Linux RH 6.0 Boot	4
Windows XP Install	361
SPECint2000 (train)	279

Προσομοίωση Αρχιτεκτονικής (1)

- Απαιτήσεις
 - Γενικότητα (Generality)
 - » Μπορεί το εργαλείο να αναλύσει τα workloads?
 - » Parallel Systems, Multithreading, Multiple address spaces, OS code, Network Systems, etc.
 - Πρακτικότητα (Practicality)
 - » Μπορεί το εργαλείο να χρησιμοποιηθεί αποδοτικά;
 - » Host assumptions, compiler assumptions, OS modifications, workload language assumptions
 - Εφαρμοσιμότητα (Applicability)
 - » Μπορεί το εργαλείο να απαντήσει στα ερωτήματά μας;
 - » Restricted state that can be monitored, restrictions on parameter visibility, restricted length of observations.

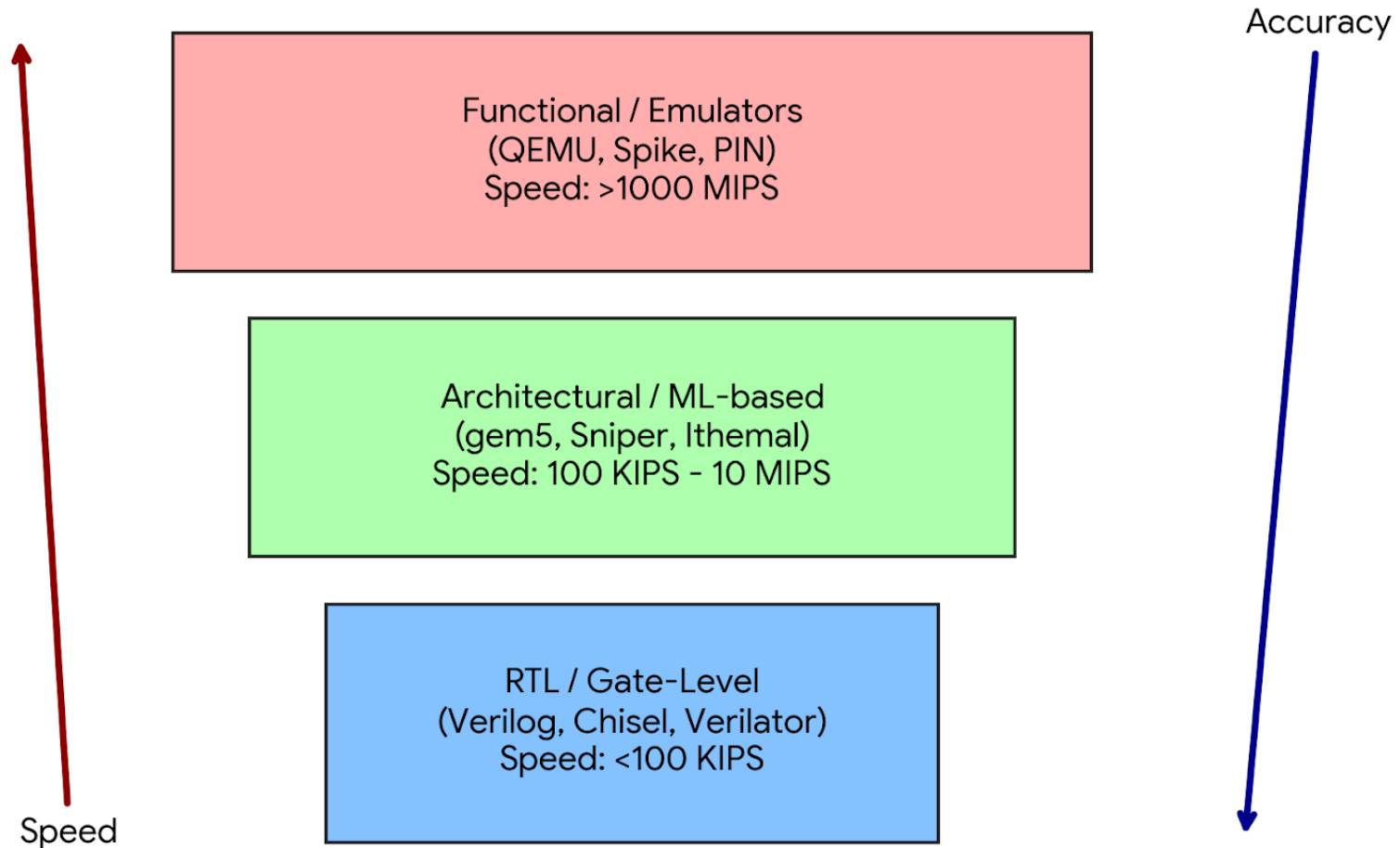
Προσομοίωση Αρχιτεκτονικής (2)

- Πλεονεκτήματα
 - Early availability
 - Ευκολία χρήσης
 - » Πλήρης διαφάνεια και ευκολία παρακολούθησης και μετρήσεων
 - » Διαφορετικά επίπεδα λεπτομέρειας και ακρίβειας
 - Pipelines, caches, branch predictors, ...
 - Hardware devices (timer, drives, cards, ...)
 - » Έλεγχος καινοτόμων προτάσεων/ιδεών
 - Κόστος
 - » Open source (Free)
 - » Academic licenses (Free ή μικρό κόστος για support)

Προσομοίωση Αρχιτεκτονικής (3)

- Προκλήσεις
 - Χρόνος ανάπτυξης των μοντέλων (modeling time)
 - Έλεγχος ορθότητας μοντέλων (validation)
 - Ταχύτητα προσομοίωσης
- Ενεργό ερευνητικό πεδίο
- Πληθώρα επιλογών
 - ~~- Virtutech WindRiver~~ Intel Simics (x86, SPARC, MIPS, Leon, ARM...)
 - ~~- AMD SimNow (x86)~~
 - SimpleScalar (Alpha, PISA, ARM, x86) [~1990-2013]
 - SimFlex [2005-2016], MARSSx86 [2010-2013]
 - SniperSim [~2010 – still alive (probably)]
 - gem5 (event driven, full/non-full system, co-simulation with SystemC <https://github.com/gem5/gem5>)
 - ChampSim (trace-based <https://github.com/ChampSim/ChampSim>)

Προσομοίωση Αρχιτεκτονικής (4)



Επιλογή περιβάλλοντος προσομοίωσης

- Κριτήρια Επιλογής
 - Modularity simulator
 - Extensibility simulator
 - Επίπεδο ακρίβειας simulator
 - Ταχύτητα simulator
 - Μέγεθος του design space που θέλουμε να μελετήσουμε
 - Επιλογή κατάλληλων benchmarks

Αποτελέσματα Προσομοίωσης(1)

- Ο σκοπός ενός timing simulation είναι η συγκέντρωση πληροφοριών και μέτρηση διαφόρων μεγεθών
 - IPC
 - Memory access cycles
 - On-chip network contention
- Τα προγράμματα παρουσιάζουν διαφορετικές φάσεις
 - Initialization phase
 - Main phase
 - Wrap-up phase
- Πότε παίρνουμε τα στατιστικά που μας ενδιαφέρουν;

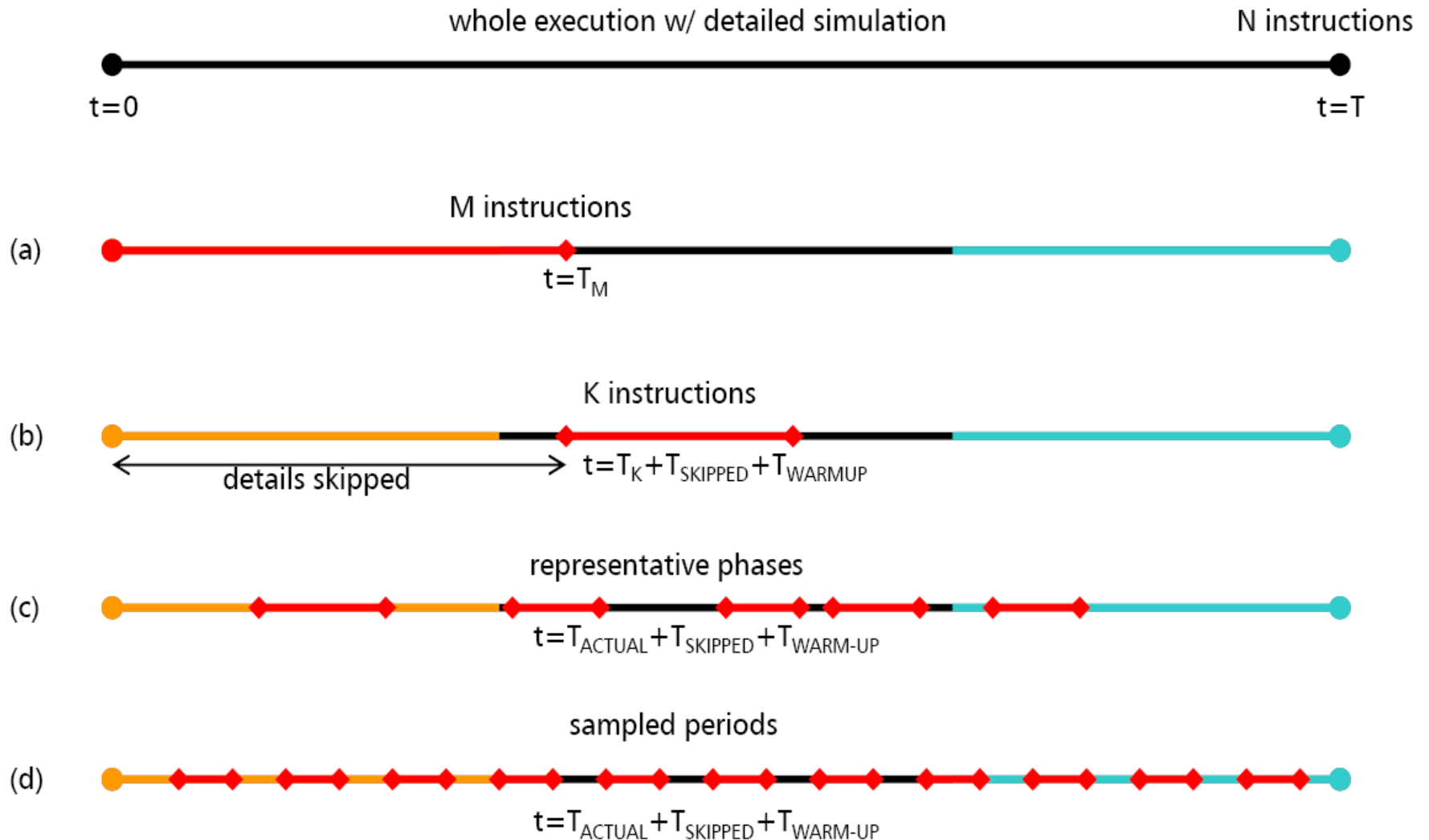
Αποτελέσματα Προσομοίωσης(2)

- Η προσομοίωση είναι ένα single-thread process
 - Ακόμα και αν προσομοιώνουμε ένα παράλληλο σύστημα
- Η ταχύτητα προσομοίωσης δε βελτιώνεται πια με την τεχνολογία
 - Παράλληλα συστήματα τρέχουν πολλαπλές προσομοιώσεις ταυτόχρονα.
- Δεν μπορούμε να φτιάξουμε γρήγορους simulators
 - Δεν προσομοιώνουμε ολόκληρο το σύστημα
 - Δεν προσομοιώνουμε ολόκληρη την εφαρμογή

Αποτελέσματα Προσομοίωσης(3)

- Προσομοίωση μικρότερου συστήματος
 - π.χ. 16 cores αντί για 1024
 - Δεν εκθέτει θέματα κλιμακωσιμότητας (scalability)
 - » Ανταγωνισμός για shared resources
 - » Conflicts
 - » Race conditions
- Προσομοίωση μικρότερης εφαρμογής
 - π.χ. Matrix multiply 1K x 1K αντί για 1M x 1M
 - Δεν εξετάζει τα όρια του hardware
 - » Το working set χωράει στις caches
 - » Capacity/conflict issues
 - » Λιγότερη επαναχρησιμοποίηση data/code
 - » Initialization vs. steady state

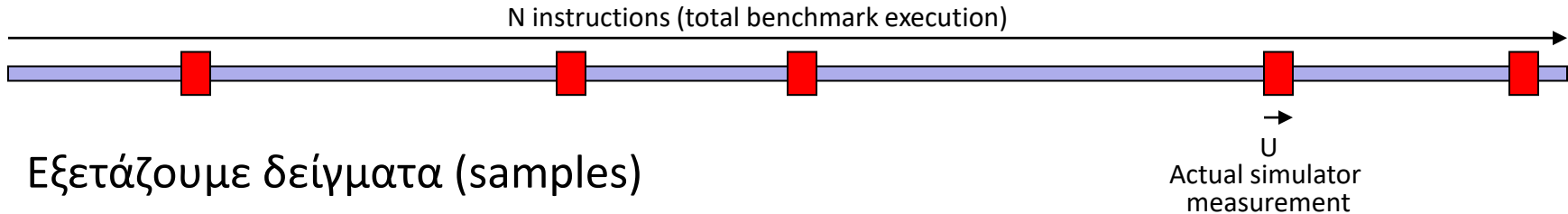
Τεχνικές Προσομοίωσης (1)



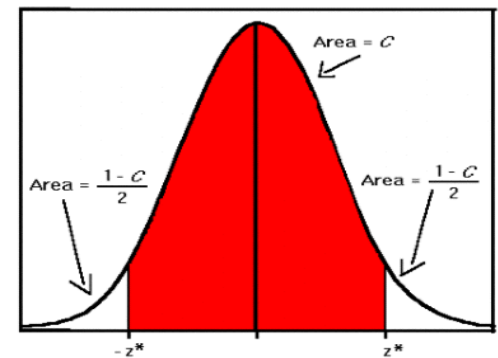
Τεχνικές Προσομοίωσης – Sampling (2)

- Sampling
 - Προσομοίωση τυχαίων σημείων της εφαρμογής
- Fast-forwarding
 - Μέχρι να φτάσουμε στο sample point
- Warm-up
 - Γρήγορη προσομοίωση πριν τη φάση των μετρήσεων
- Checkpointing
 - Αποθήκευση architectural state πριν το sample
- Phase detection
 - Επιλογή των samples μετά από ανάλυση της εφαρμογής

Τεχνικές Προσομοίωσης – Sampling (2)



- Εξετάζουμε δείγματα (samples)
- Μαθηματική προσέγγιση
 - confidence margin (eg. 95%)
 - confidence interval (eg. +/- 2.5)
- Δυο προσεγγίσεις σχετικά με την επιλογή δειγμάτων
 - Systematic sampling (π.χ. sample every N instructions)
 - Random sampling
 - Non-random sampling (phase detection, simpoints)
- Μεθοδολογία Προσομοίωσης:
 1. Fast-forwarding
 2. Warm-up (caches, branch predictors, TLBs, OS)
 3. Checkpointing
 4. Μέτρηση



Ενορχήστρωση - Instrumentation

- Εισαγωγή επιπλέον κώδικα στην εφαρμογή με στόχο την συλλογή πληροφοριών για την εκτέλεση (cache misses, total instructions executed etc.)
 - Ανάλυση προγράμματος: Performance profiling, error detection, capture & replay
 - Μελέτη Αρχιτεκτονικής: Processor & cache simulation, trace collection
 - Binary translation: Emulate unsupported instructions, modify program behavior

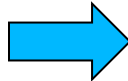
Τεχνικές Ενορχήστρωσης

- Source instrumentation
 - Εισαγωγή εντολών στον πηγαίο κώδικα της εφαρμογής
- Binary instrumentation
 - Εισαγωγή εντολών στο εκτελέσιμο της εφαρμογής
 1. Static binary instrumentation
 - » Εισαγωγή των εντολών **πριν** την εκτέλεση
 2. Dynamic binary instrumentation
 - » Εισαγωγή των εντολών **δυναμικά κατά την εκτέλεση**

Source Instrumentation – Παράδειγμα (1)

Αρχικό πρόγραμμα

```
void foo() {  
    bool found = false;  
    for (int i=0; i<100; ++i) {  
        if (i==50) break;  
        if (i==20) found=true;  
    }  
  
    printf("foo\n");  
}
```



Instrumented

```
char inst[5];  
void foo() {  
    bool found = false; inst[0]=1;  
    for (int i=0; i<100; ++i) {  
        if (i==50) {inst[1]=1; break;}  
        if (i==20) {inst[2]=1; found=true;}  
        inst[3]=1;  
    }  
  
    printf("foo\n");  
    inst[4]=1;  
}
```

Binary Instrumentation – Παράδειγμα (2)

Total Instructions counter

```
counter++;  
Sub    $0xff, %edx  
counter++;  
cmp    %esi, %edx  
counter++;  
jle    <L1>  
counter++;  
mov    $0x1, %edi  
counter++;  
add    $0x10, %eax
```

Binary Instrumentation – Παράδειγμα (3)

Instruction Trace

```
Print(ip);  
sub    $0xff, %edx  
Print(ip);  
cmp    %esi, %edx  
Print(ip);  
jle    <L1>  
Print(ip);  
mov    $0x1, %edi  
Print(ip);  
add    $0x10, %eax
```

Πλεονεκτήματα Ενορχήστρωσης

- Binary Instrumentation
 - Language independent
 - Machine-level view
 - Instrument legacy/proprietary software
- Dynamic Instrumentation
 - Δεν χρειάζεται recompile/relink
 - Εντοπισμός κώδικα κατά την εκτέλεση
 - Δυνατότητα χειρισμού κώδικα που παράγεται δυναμικά
 - Δυνατότητα ενορχήστρωσης running processes

Intel's PIN



- Εργαλείο για dynamic binary instrumentation.
- Αναπτύσσεται και συντηρείται από την Intel.
- Περισσότερες πληροφορίες:

<https://software.intel.com/en-us/articles/pin-a-dynamic-binary-instrumentation-tool>

- Χαρακτηριστικά/Πλεονεκτήματα
 - Εύκολη εντοχόστρωση
 - Programmable (APIs)
 - Multiplatform (x86, x86-64, Windows, Linux, OSX, Android)
 - Robust
 - » Real-life apps: Database, web browseres, ...
 - » Multithreaded apps
 - » Signals
 - Υψηλής Απόδοσης (compiler optimizations on instrumentation code)
 - Ευρεία αποδοχή από την ακαδημαϊκή κοινότητα και την βιομηχανία

PIN – Χρήση

Download latest version:

<https://software.intel.com/en-us/articles/pintool-downloads>

Unzip:

```
$ tar xvfz pin-3.30-98830-g1d7b601b3-gcc-linux.tar.gz
$ cd pin-3.30-98830-g1d7b601b3-gcc-linux
$ ls -aF
./ ../ doc/ extras/ ia32/ intel64/ LICENSE pin pin.sh README redist.txt
source/
```

Execute:

```
$ ./pin.sh -t pintool - application
```

ή

```
$ ./pin.sh -t pintool -pid 1243
```


PIN – Παράδειγμα

```
$ ./pin.sh -t inscount0.so -- /bin/ls
```



Total instructions count pintool

```
$ cd source/tools/ManualExamples/  
$ make inscount0.so  
$ ls obj-intel64/inscount0.so
```

Pintools

- Προγράμματα γραμμένα σε C++
- Χρησιμοποιούνται κατά την εκτέλεση του PIN για την ανάλυση της εφαρμογής
- Χρησιμοποιούν το PIN API:
 - » `INS_AddInstrumentFunction(...);`
 - » `INS_InsertCall(...);`
 - » ...
- PIN API reference:
 - https://software.intel.com/sites/landingpage/pintool/docs/55942/Pin/html/group__API__REF.html

Pintool code

```
#include <iostream>
#include "pin.h"
```

```
UINT64 icount = 0;
```

```
void docount() {
    icount++;
}
```

analysis routine

```
void Instruction(INS ins, void *v) {
    INS_InsertCall(ins, IPOINT_BEFORE, (AFUNPTR) docount, IARG_END);
}
```

instrumentation routine

```
void Fini(INT32 code, void *v){
    std::cerr << "Count " << icount << "\n";
}
```

end routine

```
int main(int argc, char *argv[]) {
    PIN_Init(argc, argv);
    INS_AddInstrumentFunction(Instruction, 0);
    PIN_AddFiniFunction(Fini, 0);
    PIN_StartProgram();
    return 0;
}
```

Δυνατότητες Pin & Pintools

- Αντικατάσταση των application functions.
- Εντοπισμός/καταγραφή/έλεγχος κάθε application instruction.
- Πέρασμα παραμέτρων στο instrumentation routine
 - register values, register values by reference (for modification)
 - memory addresses read/written
 - full register context
- Εντοπισμός syscalls και αλλαγή arguments
- Εντοπισμός/έλεγχος application threads
- ...
- ...